Privacy Preserving KYC

Mid Term Project Report

Initial Prototype Implementation

Course: CS670, Cryptographic Techniques for Privacy Preservation

Team Members

Vundela Obula Reddy
Kishore Senthilkumar
Cezan Vispi Damania
Srijani Gadupudi

Department of Computer Science and Engineering
IIT Kanpur

September 12, 2026

1 Application Outline

Building on our literature survey, we have started implementing a privacy-preserving KYC application that enables verification without directly revealing the user's personal information.

The application consists of three entities: the **user**, **issuer**, and **verifier**. The issuer verifies and issues the KYC credential, the user holds the credential, and the verifier specifies the required KYC conditions.

The basic data flow of the application is shown below.

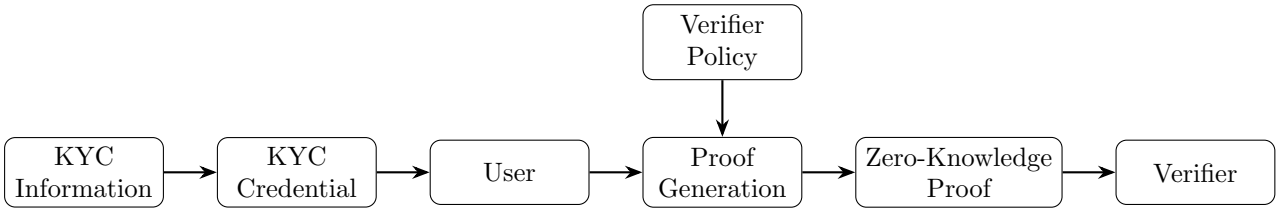


Figure 1: Initial architecture and data flow of the Privacy Preserving KYC system.

2 Initial Implementation

2.1 KYC Data and Credential Representation

We represent the user's KYC information as a structured collection of attributes. Each attribute is stored as a key-value pair.

```
struct KYCAttribute {
    string name;
    string value;
};

struct KYCData {
    vector<KYCAttribute> attributes;
};
```

Typical attributes include age, nationality, address, and identity status. The KYC data remains private. The credential stores the verified attributes separately from the verifier's policy, allowing the same credential to support different verification requests. The policy is passed to the proof-generation component without revealing the user's actual age.

2.2 Zero-Knowledge Proof Component

We have started implementing the zero-knowledge proof component using **Circom** for circuit construction and **snarkjs** with the **Groth16** proving system for proof generation and verification. Circom allows us to express KYC requirements as mathematical constraints, while snarkjs handles witness generation, proof generation, and verification.

The initial prototype focuses on an age-based KYC condition:

$$age \geq threshold$$

Here, the user's actual age is a **private input**, while the required age threshold is a **public input**. For example, the user may provide an age of 23 while the verifier specifies a threshold of 18.

The proof-generation pipeline is:

```
KYC Data -> Circom Circuit -> Witness
         -> Groth16 Prover -> ZK Proof
         -> Groth16 Verifier -> Valid/Invalid
```

The verifier receives the proof and public inputs but does not learn the user's actual age. Once the basic age-check circuit is functional, the same approach will be extended to other KYC attributes and combined policies.

3 Team Contributions

Vundela Obula Reddy and Kishore Senthilkumar – KYC Data and ZKP Implementation

- Working on the KYC attribute structures and their mapping to ZKP inputs.
- Working on Circom circuits for conditions such as $age \geq threshold$.
- Working on compiling circuits to R1CS and WASM.
- Working on witness generation using `snarkjs`.
- Working on Groth16 key setup, proof generation, and verification.

Cezan Vispi Damania and Srijani Gadupudi – Application Architecture and Privacy

- Working on the frontend–backend structure for user, issuer, and verifier interactions.
- Working on the KYC credential model and issuer-to-user credential flow.
- Working on the policy representation used to specify verifier requirements.
- Working on the API interface connecting the application layer with the ZKP module.
- Working on keeping private KYC attributes on the user side and sending only proofs and required public inputs to the verifier.

4 Relation to the Literature Survey

The implementation work is guided by the techniques identified in our literature survey.

The *zkKYC* work motivates our use of zero-knowledge proofs to verify KYC conditions without directly revealing personal information.

Self-Sovereign Identity systems motivate our issuer–user–verifier structure and credential-based representation of verified identity.

Camenisch–Lysyanskaya credentials motivate our credential layer and its potential for selective disclosure.

Bulletproofs provide an alternative approach for proving numerical KYC conditions such as range and threshold checks.

CP-ABE, MPC, Homomorphic Encryption, and blockchain-based KYC systems provide possible directions for future extensions.

5 Next Steps

The main priority now is to continue developing and connecting the basic components of the system.

- Complete the application architecture for user, issuer, verifier, credential, policy, and ZKP components.
- Integrate the Circom circuits with the application and connect Groth16 proof generation and verification.
- Extend the ZKP circuits to support additional KYC conditions.
- Test that raw KYC attributes are not exposed to the verifier and that only required public inputs and proofs are accessible.
- Test the complete workflow with valid and invalid KYC inputs and verify the correctness of the generated proofs.